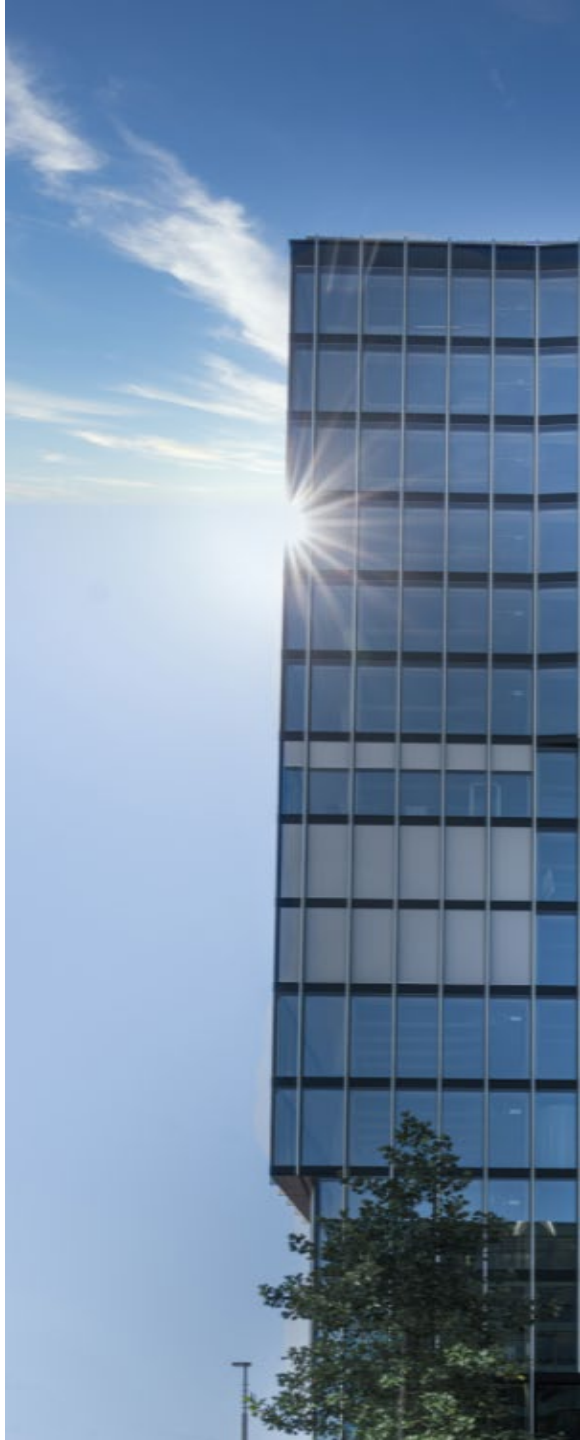
HSLU Hochschule Luzern

Objektorientierte Programmierung

Datenstrukturen: Collections - Einsatz

Roland Gisler



FH Zentralschweiz

Inhalt

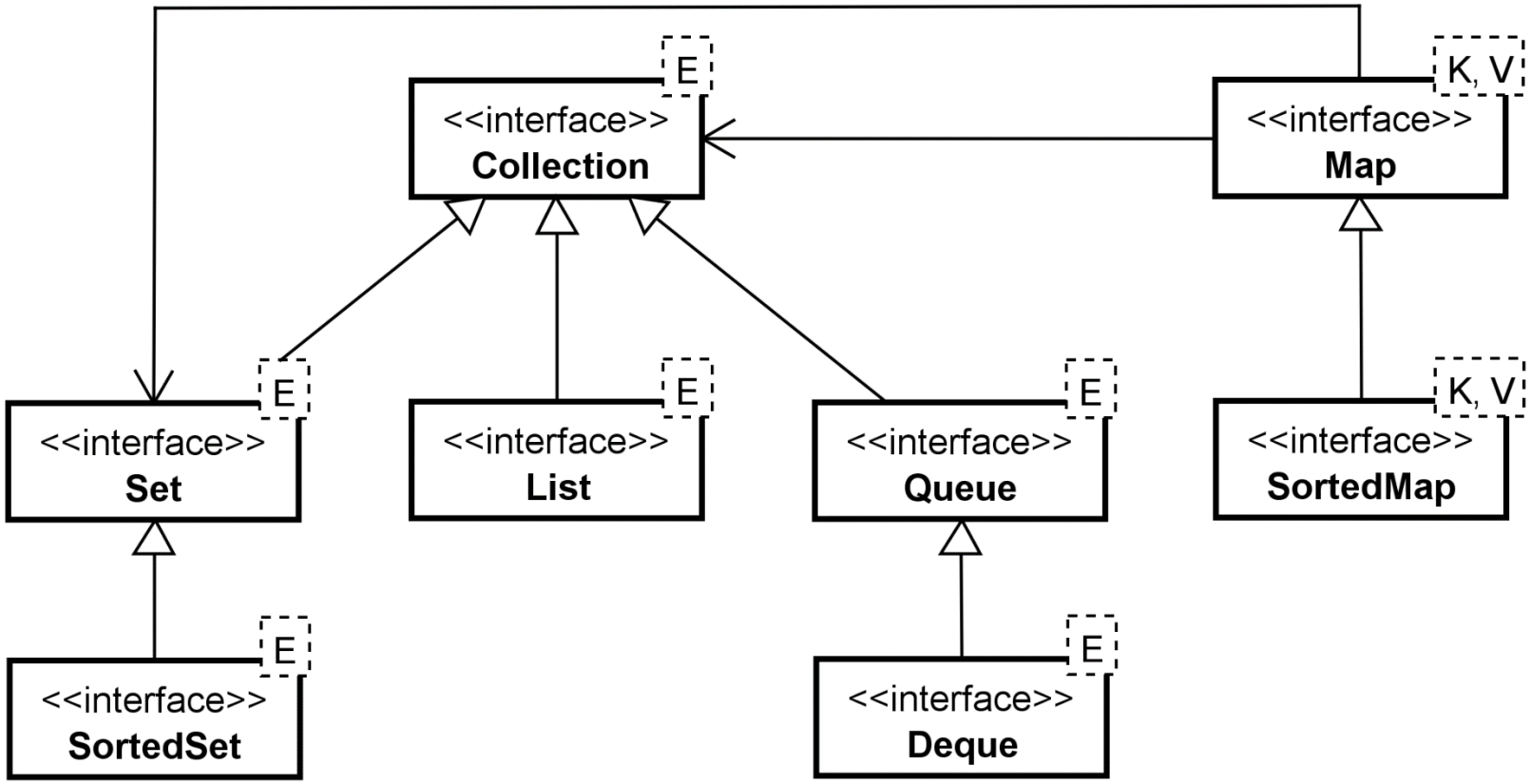
- Überblick: Collection Interfaces
- Basis-Interface: `Collection`
- Ausgewählte Collection-Interfaces und Implementationen:
  - `List`, `Set`, `Queue`, `Deque`, `Map`
- Konzept des Iterators
- Verwendung und Manipulation von Datenstrukturen
- Zusammenfassung

Lernziele

- Sie kennen einige ausgewählte Datenstrukturen und ihre Semantik.
- Sie verstehen das Konzept des Iterators.
- Sie können Iteratoren auf verschiedenen Datenstrukturen nutzen.
- Sie können ausgewählte Datenstrukturen praktisch nutzen und einfache Operationen auf/mit diesen durchführen.

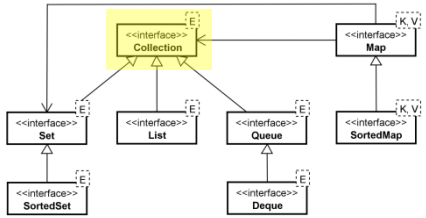
Java Collection Framework – Interfaces - Übersicht

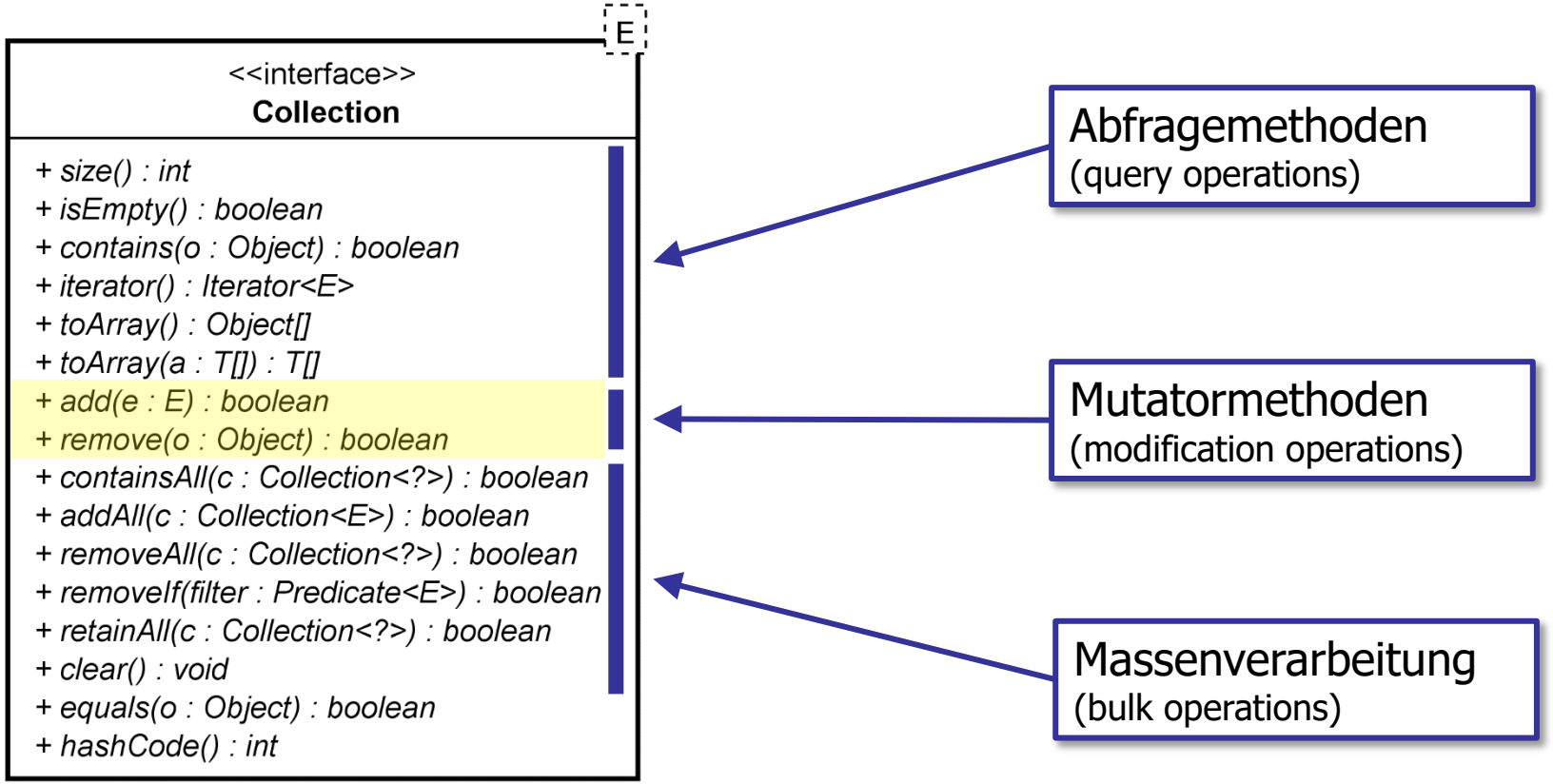
- Auszug aus den vorhandenen Schnittstellen (nicht vollständig):



Interface `Collection`

Interface `Collection`: Die Basis

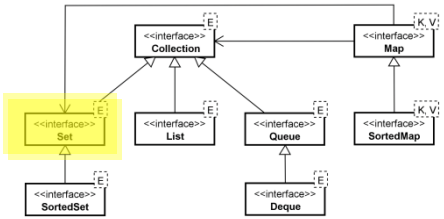


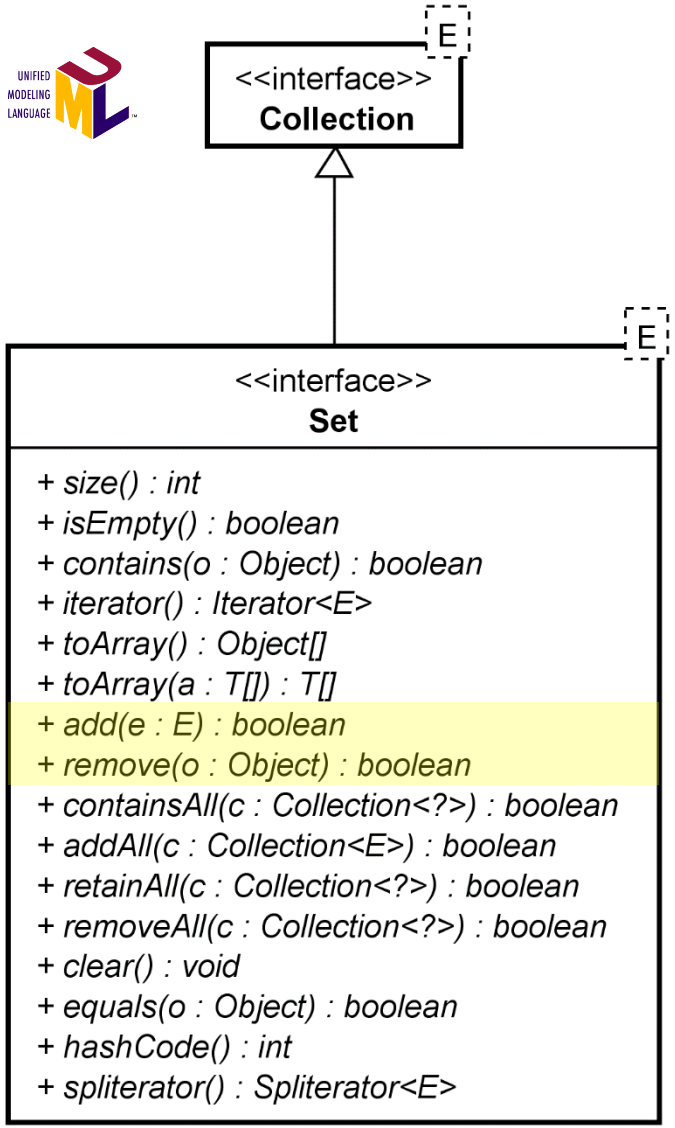


- Dieses «Muster» ist typisch für fast alle Datenstrukturen.
  - ➔ Kann eine Orientierungshilfe sein.

Interface `Set`

Interface `Set<E>`





- Sehr ähnlich zur **`Collection`**.
  (Es ist ja eine Spezialisierung!)
- Die veränderte **Semantik** ist wichtig:
  Es sind **keine Duplikate** (im Sinne von **`equals`**!) erlaubt!

`Set<E>` - Eigenschaften

- Definiert eine Menge von Objekten des Typs `E`.
- Jedes Objekt kann nur **ein einziges Mal** enthalten sein.
  - Gleichheit wird per `equals()` festgelegt.
  - Spezialfall: `null` ist erlaubt (aber natürlich nur einmal!)
- Objekte, die in einem Set eingefügt sind, sollten **nicht** mehr **verändert** werden!
  - Durch die Veränderung könnten identische Objekte entstehen, was die Semantik des Sets verletzen würde!
  - Empfehlung: Objekte für/vor Veränderungen aus Datenstruktur **entnehmen** und danach wieder **neu einfügen**!
- Implementationen (Beispiele):
  `HashSet`, `LinkedHashSet`, `TreeSet`, `EnumSet` etc.

Set – Beispiel mit `HashSet<E>`

```java
final Set<Temperatur> verlauf = new HashSet<>();

verlauf.add(new Temperatur(10.2f));
verlauf.add(new Temperatur(15.8f));
verlauf.add(new Temperatur(21.3f));

System.out.println("Anzahl Messwerte: " + verlauf.size());
```

```
Anzahl Messwerte: 3
```

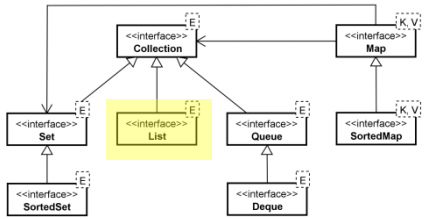
```java
final boolean result = verlauf.add(new Temperatur(15.8f));
System.out.println("Anzahl Messwerte: " + verlauf.size());
```

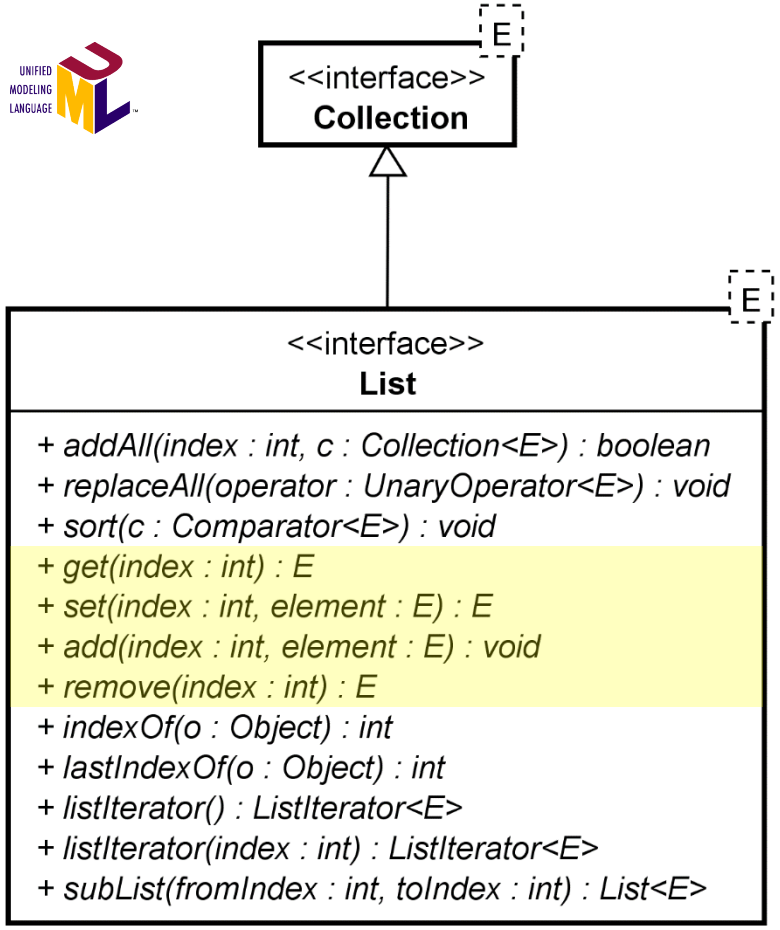
```
Anzahl Messwerte: 3
```

- Der Versuch ein Duplikat einzufügen, misslingt und liefert `false` zurück:
  Es somit weiterhin nur **drei** Werte enthalten.

Interface `List`

Interface `List<E>`





- Ergänzend zur **`Collection`** kann bei der **`List`** über einen Index **direkt** auf die einzelnen Elemente zugegriffen werden.
- **`List`** hat somit nichts mit einer klassischen «Liste» (einfach oder doppelt verknüpft) zu tun, sondern erinnert eher an einen Array.

`List<E>` - Eigenschaften

- Geordnete Datenstruktur von Elementen vom Typ `E`.
  - Geordnet: Elemente behalten ihre (Einfüge-)Position bei, sind aber nicht sortiert.
- Eine Liste kann mehrmals das gleiche Objekt enthalten.
- Die Nutzer*in hat über den ➔Index die Kontrolle, an welcher Stelle der Liste die Elemente eingefügt werden (Position in der Liste).
- Zero-based Index: Das erste Element hat den Index `0`.
  - Vergleichbar mit fixed-size Arrays, z.B. `int[0]`
- Beispiele für Implementationen:
  - `ArrayList`, `LinkedList`, `Stack`*, `Vector`* etc.

* Hinweis: Das sind synchronisierte und darum langsamere Klassen! Es sind schnellere (unsynchronisiert) und somit oft «bessere» Alternativen vorhanden: `Stack` ➔ z.B. `Deque`-Implementationen, `Vector` ➔ z.B. `ArrayList`)

List – Beispiel mit `ArrayList<E>`

```java
final Collection<Temperatur> verlauf = new ArrayList<>();

verlauf.add(new Temperatur(10.2f));
verlauf.add(new Temperatur(15.8f));
verlauf.add(new Temperatur(21.3f));
System.out.println("Anzahl Messwerte: " + verlauf.size());
```

```
Anzahl Messwerte: 3
```

```java
((List<Temperatur>) verlauf).set(1, new Temperatur(18.5f));
final Temperatur t2 = ((List<Temperatur>) verlauf).get(1);
System.out.println("2. Messwert: " + t2.getCelsius());
```

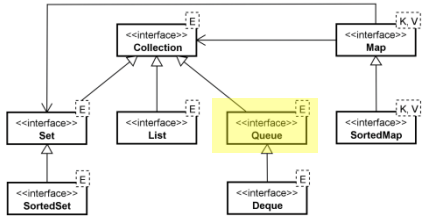
```
2. Messwert: 18.5
```

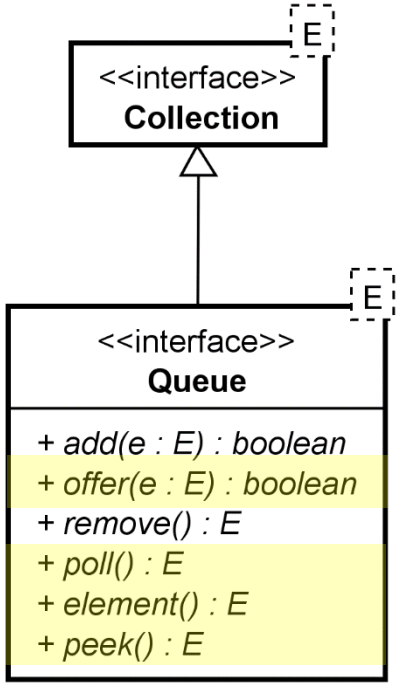
- Hinweis: Für **`add()`** reicht der Typ **`Collection`**, erst für die Methoden **`set(…)`** und **`get(…)`** benötigt man den Typ **`List`**.

➔ Casting wird hier exemplarisch demonstriert (**nicht** elegant)!

Interface `Queue`

Interface `Queue<E>`





- Spezialisiert die Semantik der **`Collection`** zu einer (Warte-)Schlange: Meistens nach FIFO-Semantik (**f**irst **i**n, **f**irst **o**ut), kann aber auch priorisiert sein.
- Typische Methoden(-namen):
  - **`offer()`** – Hängt ein Element am Ende an.
  - **`poll()`** – Entnimmt ein Element am Anfang.

`Queue<E>` - **Eigenschaften**

- Definiert eine Queue von Elementen vom Typ **`E`**.
  - Eine Queue kann eine statische, beschränkte Grösse haben.
  - Elemente werden am Schwanz (tail) angehängt.
  - Elemente werden am Kopf (head) entnommen.
- Typischer Verwendungszweck: Zwischenspeichern (puffern) von Objekten vor ihrer Verarbeitung.
  - Beispiele: Warteschlange, Buffer, Unix-Pipe, Druckerqueue.
- Semantik meist nach FIFO-Prinzip (**f**irst **i**n, **f**irst **o**ut).
  - Variante: Priority-Queue sortiert automatisch nach Priorität.
- Beispiele für Implementationen:
  `ArrayDeque`, **`DelayQueue`**, **`LinkedBlockingQueue`**, **`PriorityQueue`** etc.

Queue – Beispiel mit `ArrayBlockingQueue<E>`

```java
final Queue<Temperatur> queue = new ArrayBlockingQueue<>(5);
queue.offer(new Temperatur(10.2f));
queue.offer(new Temperatur(15.8f));
queue.offer(new Temperatur(21.3f));
System.out.println("Anzahl Messwerte: " + queue.size());

System.out.println("1. Wert: " + queue.poll());
System.out.println("2. Wert: " + queue.poll());
System.out.println("3. Wert: " + queue.poll());
```

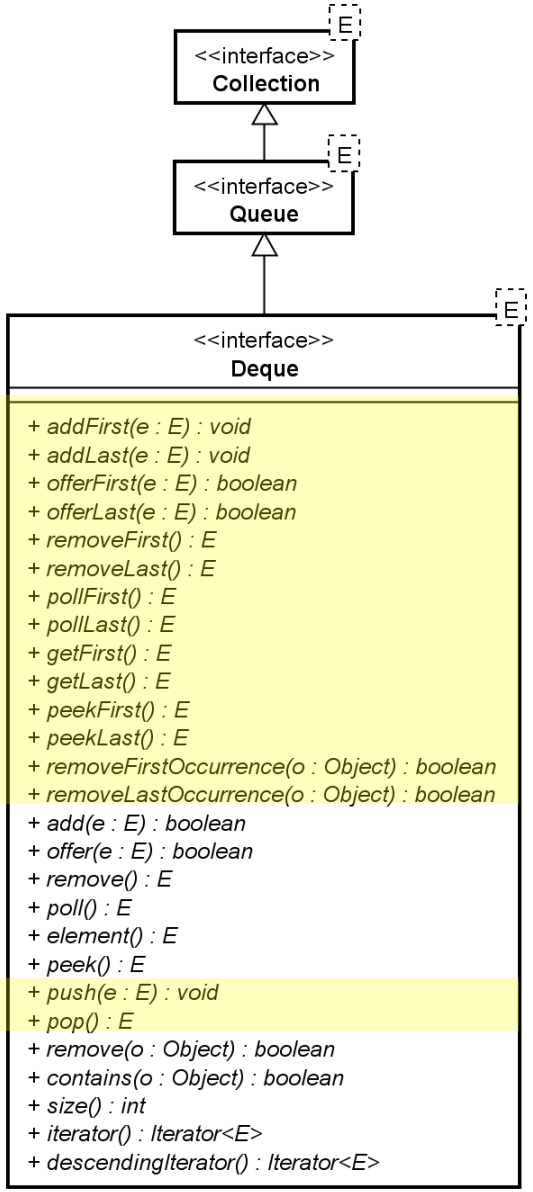
```
Anzahl Messwerte: 3
```

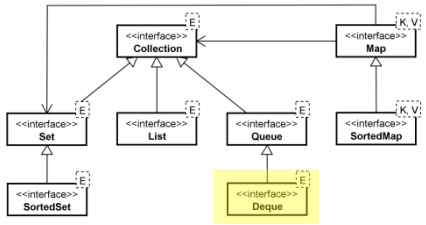
```
1. Wert: Temperatur[kelvin=283.3;celsius=10.2]
2. Wert: Temperatur[kelvin=289.0;celsius=15.8]
3. Wert: Temperatur[kelvin=294.5;celsius=21.3]
```

- Beispiel der **`ArrayBlockingQueue`**-Implementation: Blockiert (wartet) bei **`offer()`** wenn die Queue voll ist und bei **`poll()`** wenn die Queue leer ist.

Interface `Deque`

Interface `Deque<E>`





- Spezialisiert die Semantik der `Collection` zu einer Queue, welche in **beide** Richtungen verwendet werden kann.
- Das ermöglicht unter anderem auch einen **Stack**: FILO-Semantik (**f**irst **i**n, **l**ast **o**ut), was einen klassischen «Stapelspeicher» repräsentiert.
- Empfohlene Methoden für Stack:
  - `push()` – Element auf Stack legen.
  - `pop()` – Element von Stack abheben.

`Deque<E>` - Eigenschaften

- Definiert eine Deque von Elementen vom Typ `E`.
  - Deque ist eine Abkürzung: «**D**ouble **E**nded **QUE**ue».
  - Somit eine Queue, welche in beide Richtungen arbeiten kann.
- Semantik kann somit wahlweise nach FIFO-Prinzip (in beide Richtungen) oder nach FILO-Prinzip (an beiden Enden) genutzt werden
  - Semantik ist somit abhängig von der konkreten Verwendung!
- Typischer Verwendungszweck: Zwischenspeichern von Objekten vor/für die Verarbeitung, aber mit flexibler Semantik.
- Beispiele für Implementationen: `ArrayDeque`, `ConcurrentLinkedDeque`, `LinkedBlockingDeque`, `LinkedList` etc.

Deque – Beispiel mit `ArrayDeque<E>`, hier genutzt als Stack

```java
final Deque<Temperatur> stack = new ArrayDeque<>(5);
stack.push(new Temperatur(10.2f));
stack.push(new Temperatur(15.8f));
stack.push(new Temperatur(21.3f));
System.out.println("Anzahl Messwerte: " + stack.size());

System.out.println("3. Wert: " + stack.pop());
System.out.println("2. Wert: " + stack.pop());
System.out.println("1. Wert: " + stack.pop());
```

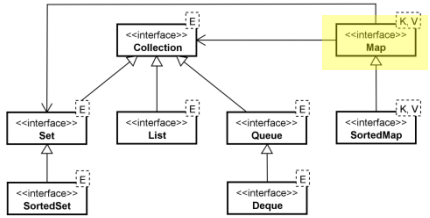
```
Anzahl Messwerte: 3
```

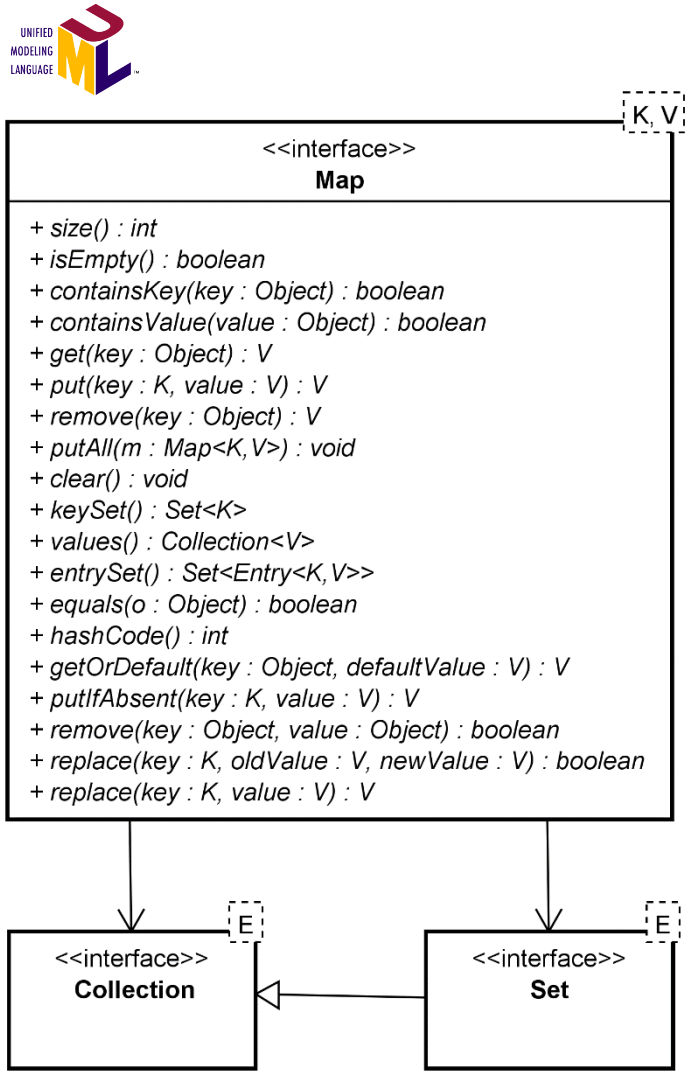
```
3. Wert: Temperatur[kelvin=294.5;celsius=21.3]
2. Wert: Temperatur[kelvin=289.0;celsius=15.8]
1. Wert: Temperatur[kelvin=283.3;celsius=10.2]
```

- Exemplarische Nutzung der **`ArrayDeque`** als **Stack**: Die FILO-Semantik führt zu einer Umkehr der Reihenfolge der Elemente.
- **Wichtig**: Unbedingt die semantisch sinnvollen Methoden nutzen!

Interface `Map`

Interface `Map<K,V>`





- Spezielle Datenstruktur für Schlüssel-Wert-Paare (**k**ey-**v**alue pairs):
  Erlaubt die Ablage eines Elementes (**`value`**) unter einem eindeutigen Schlüssel (**`key`**).
- Beispiele:
  - Personalnummer und Personenobjekt.
  - Symbol und chemisches Element.

`Map<K,V>` - Eigenschaften

- Definiert eine Map mit Schlüssel vom Typ `K` (**K**ey) und für Werte vom Typ `V` (**V**alue).
- Ein **Schlüssel** muss innerhalb einer Map **eindeutig** sein.
  - Schlüssel werden intern in einem `Set<K>` abgelegt.
- Sowohl Wert als auch Schlüssel dürfen `null` sein (Schlüssel aber pro Map nur einmal). Ein Wert-Objekt kann mehrfach in einer Map abgespeichert werden (unter verschiedenen Schlüsseln).
- Schlüssel-Objekte in einer Map sollen nach dem Einfügen nicht verändert werden!
  - Tipp: Für Bearbeitung entnehmen und danach neu einfügen.
- Beispiele für Implementationen:
  `EnumMap`, `HashMap`, `Hashtable`, `Properties`, `TreeMap` etc.

Map – Beispiel mit `HashMap<K,V>`

```
final Map<String, Temperatur> map = new HashMap<>();
map.put("first", new Temperatur(10.2f));
map.put("second", new Temperatur(15.8f));
map.put("third", new Temperatur(21.3f));
System.out.println("Anzahl Messwerte: " + map.size());
```

```
Anzahl Messwerte: 3
```

```
map.put("second", new Temperatur(22.6f));
System.out.println("2. Wert: " + map.get("second"));
```

```
2. Wert: Temperatur[kelvin=295.8;celsius=22.6]
```

- Die Schlüssel dürfen (als **`Set`**) keine Duplikate enthalten. Darum wird mit dem zweiten **`put()`** mit dem Key "`second`" der bereits enthaltende Value (das **`Temperatur`**-Objekt) **ersetzt**.
- **Hinweis**: `String` als Typ für den Key hier beispielhaft und **sehr** ineffizient!

Unterschiedliche Implementationen

Unterschiedliche Implementationen von Collections

- Für alle Collection Interfaces gibt es typischerweise mehrere, verschiedene Implementationen:
  - Allgemeine, universelle Implementationen.
  - Implementationen für nebenläufige Zugriffe.
  - Wrapper-Implementationen für/mit Zusatzfunktionalität.
  - Implementationen für Spezialfälle.
- Studieren der Dokumentation (und Erfahrung) ist wichtig, um die optimale Implementation für die jeweilige Situation auszuwählen.
- Achtung: Viele Implementationen sind **nicht synchronisiert**, d.h. nicht für nebenläufige Zugriffe geeignet (➔dafür aber schneller).
- Jede Implementation kann aber auch synchronisiert verwendet werden: Wrapping mit Methoden von `Collections` ➔folgt später.

Iteratoren

Alle Elemente einer Datenstruktur verarbeiten

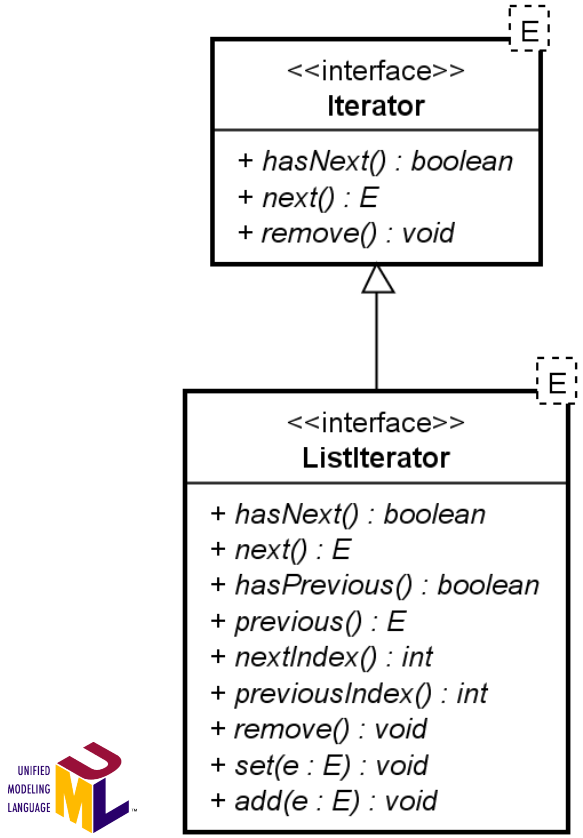
- Sehr häufig werden wir bei Elementmengen alle Elemente einzeln «durchgehen» (besuchen, traversieren) wollen.
  - z.B. zur Ausgabe, zum Suchen eines Elementes, zur Verarbeitung oder Filterung.
- Beispiel für eine **`ArrayList`** (**`List`**) mit Index und **`for`**-Loop:

```
for(int i = 0; i < list.size(); i++) {
    list.get(i).doSomething();
}
```

- **Problem**: Nicht alle Datenstrukturen erlauben einen direkten Zugriff per Index (z.B. verkettete Liste oder Baumstrukturen nicht). Trotzdem möchten wir **alle Elemente** einfach traversieren können!
- Schön wäre eine **universelle** Lösung, welche **unabhängig** von der konkreten Datenstruktur arbeitet und für alle funktioniert!

Lösung: Iterator-Konzept (Designpattern)

- **Iteratoren** erlauben das Traversieren einer Collection **ohne Bezug** auf die konkrete Art der Datenstruktur zu nehmen!
- Jedes in der Collection enthaltene Element wird während einer Iteration **genau einmal** zurückgegeben.
- Dank Polymorphie kommt jeweils der angepasste Algorithmus (➔konkreter Iterator) jeder einzelnen Datenstruktur zum Einsatz.
- Zwei Varianten von Iteratoren:
  `Iterator` – kann nur **vorwärts** traversieren.
  `ListIterator` – kann vorwärts **und** rückwärts.
  - Hinweis: `ListIterator` ist nur auf ausgewählten Datenstrukturen verfügbar.



Verwendung von Iteratoren

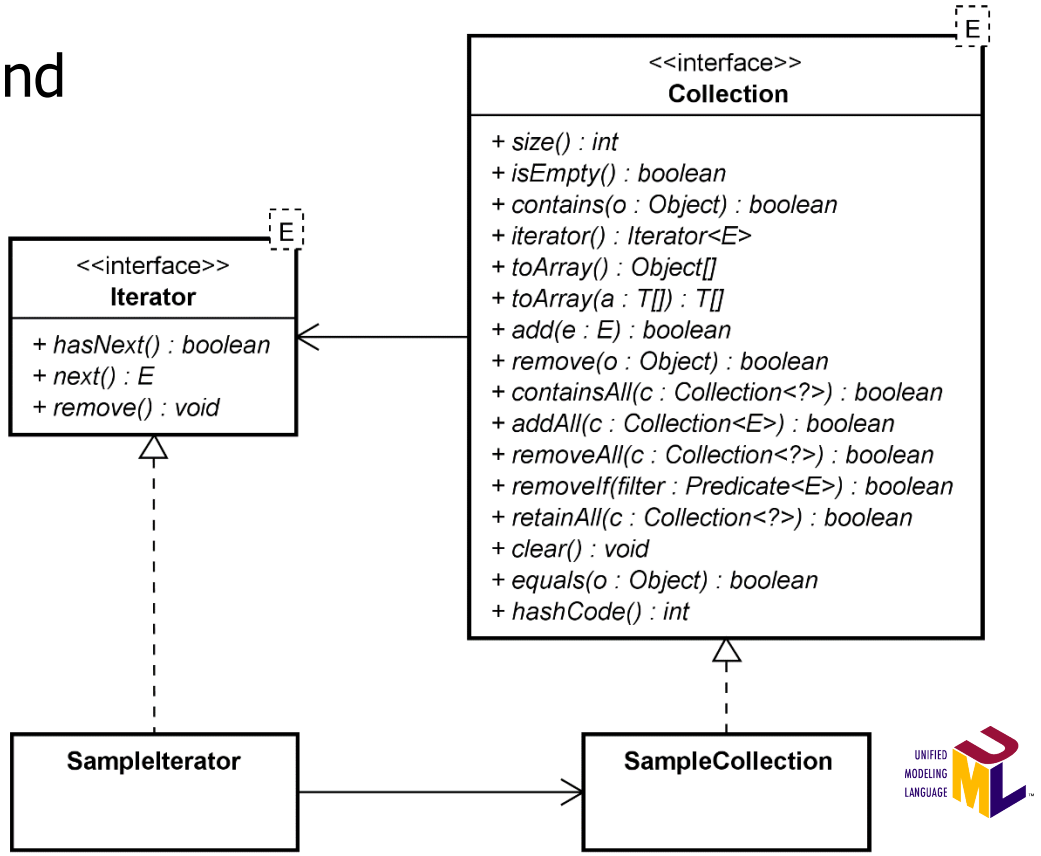
- Das Interface `Collection` bietet eine Methode `iterator()`, mit welcher man einen `Iterator` beziehen kann.
  - **Wichtig**: Für jede Iteration einen neuen Iterator beziehen!
- Beispiel (für `ArrayList<Temperatur> list`):

```
Iterator<Temperatur> iterator = list.iterator();
while(iterator.hasNext()) {
    final Temperatur t = iterator.next();
    t.doSomething();
}
```

- **Achtung**: Die Methode `next()` ist **konsumierend**, das heisst sie liefert ein Element **und** geht gleichzeitig zum nächsten Element.
  - Tipp: Referenz des Objektes für mehrfachen Gebrauch in einer lokalen Variablen zwischenspeichern (wie oben gezeigt).

Iterator-Pattern – Entwurfsmuster nach GoF

- Basiert auf einem ➔Entwurfsmuster der ➔Gang of Four (GoF).
  Mehr dazu im Modul **V**erteilte **S**ysteme und **K**omponenten (VSK).
- Die konkreten Implementationen der Datenstrukturen stellen jeweils passende Iteratoren zur Verfügung.
- Erkenntnis: Iteratoren halten den Zustand einer konkreten Iteration fest.



Vereinfachte Syntax: for[each]

- Seit Java 1.5 gibt es für fixed-size-arrays **und** Datenstrukturen, welche **Iteratoren** unterstützen, eine einheitliche, vereinfachte Syntax:

```
for(final Temperatur t : list) {
    t.doSomething();
}
```

- Diese Syntax vermeidet nicht nur typische Fehler, sondern ist auch wesentlich leichter verständlich, eleganter und typsicher.
  ➔ Verwendung sehr empfohlen!
- **Wichtig**: Der Compiler übersetzt diese Syntax in eine ganz normale, klassische `while`-Schleife mit einem `Iterator`, auf welchen man jedoch **keinen** Zugriff hat.

Hinweise zur Verwendung von Iteratoren

- Bei Verwendung von Iteratoren ist darauf zu achten, dass man den Inhalt der Datenstruktur während dessen **nicht** verändert!
  - **kein** Einfügen neuer Elemente.
  - **keine** Änderung enthaltender Elemente.
  - **kein** Entfernen enthaltener Elemente.
- Analogie: Wenn jemand eine Leiter hochklettert, verändert man die Leiter auch nicht (keine Sprossen entfernen oder verschieben)!
  - Korrekte Semantik des Iterators ist sonst nicht mehr garantiert.
- **Wichtig**: Verwendet man einen `Iterator`, um ein Element zur Entfernung zu suchen, gibt es dafür eine spezielle `remove()`-Methode **auf** dem Iterator!

Manipulation von Datenstrukturen

Manipulationen von Datenstrukturen - Sortieren

- Mittels statischer Methoden von der Klasse **`Collections`** lassen sich ausgewählte Datenstrukturen einfach **sortieren**:

```
final List<Temperatur> list = new ArrayList<>();
…
// Sortiert Liste aufsteigend (natürliche Ordnung)
Collections.sort(list);
…
// Sortiert Liste absteigend (natürliche Ordnung)
Collections.sort(list, Collections.reverseOrder());
```

- Für spezielle Sortierungen steht eine **`sort()`**-Methode zur Verfügung, welche ein **`Comparator`**-Objekt entgegen nimmt.
- **Wichtig**: Adäquate **`equals()`**- und **`hashCode()`**-Methoden und die Implementation von **`Comparable`** werden vorausgesetzt! ➔ siehe **O09_IP_ObjectEqualsCompare**

Manipulationen von Datenstrukturen – weitere Beispiele

- Es stehen noch zahlreiche weitere Funktionen zur Verfügung, hier nur ein paar ausgewählte Beispiele:

```
final List<Temperatur> list = new ArrayList<>();

// Bestimmt das grösste Objekt (über Comparable).
final Temperatur maxima = Collections.max(list);

// Bestimmt das kleinste Objekt (über Comparable).
final Temperatur minima = Collections.min(list);

// Zählt wie oft ein Objekt (hier maxima) auftritt.
final int count = Collections.frequency(list, maxima);
```

- **Wichtig**: Adäquat implementierte `compareTo()`-Methode (vom Interface `Comparable`) wird vorausgesetzt!

Zusammenfassung

- Verständnis der grundlegenden Interfaces und Semantik von Datenstrukturen.
- Beispielanwendungen von ausgewählten Implementationen.
- Konzept und Einsatz eines Iterators.
- Manipulation von Datenstrukturen.

Fragen?